

Exercise 4:

```
a = 0; % lower limit
syms f(r,R);
f(r,R) = r*2*(1-r/R)^(1/6);
R = 2; % cm and upper limit
f = subs(f);
Q_symbolic = 2*pi*int(f,r,a,R);
f1 = @(r)integrand(r,R);
Q_integral = integral(f1,a,R);
Q_simpson = simpson_onethird(f1,a,R,100);
fprintf("Results: \n" + ...
    "Flow Rate Q symbolic = %f \n" + ...
    "Flow Rate Q integral = %f \n" + ...
    "Flow Rate Q simpson 1/3 = %f",Q_symbolic, Q_integral,Q_simpson)
```

```
Results:
Flow Rate Q symbolic = 19.885246
Flow Rate Q integral = 19.885246
Flow Rate Q simpson 1/3 = 19.834620
```

Comparison:

```
p_error=100*abs(Q_simpson-Q_integral)/Q_integral;
fprintf("The result obtained from the symbolic integral, and the built in integral function is
    "However, simpsons 1/3 for 100 segments (n = 100) has a percent error of %f when compared
    "with the symbolic (analytical) and integral (built in) functions results.", p_error)
```

The result obtained from the symbolic integral, and the built in integral function is the same. However, simpsons 1/3 for 100 segments (n = 100) has a percent error of 0.254589 when compared with the symbolic (analytical) and integral (built in) functions results.

```
function [rV] = integrand(r,R)
rV = 2*pi*2*r.*(1-r/R).^(1/6);
end

function [I] = simpson_onethird(f,a,b,n)
if (mod(n,2)==0) % total number of segments is even
    I=0;
    h = (b-a)/n;
    for i=1:n
        x = a + i*h;
        if (i==0 || i==n)
            I = I + f(x); % endpoints
        elseif (mod(i,2)==0)
            I = I + 2*f(x);
        else
            I = I + 4*f(x);
        end
    end
    I = h*I/3;
else
    disp("n must be even")
end
```

end